

Gonka Objection Handling Playbook

Version: 1.0 **Date:** 2026-04-01 **Classification:** Internal – for go-to-market, community management, and developer relations teams **Feeds into:** Phase 18 (Channel Strategy), Phase 19 (Partnership & Ecosystem) **Requirement:** MSG-02 (completion)

1. Executive Summary

This playbook provides evidence-backed responses for every objection a developer raises when evaluating Gonka as their OpenClaw inference provider. It is organized by persona so you can quickly look up the developer you are talking to, find their specific objection, and use the structured response framework.

How to use this playbook:

1. Identify which persona you are speaking with: Weekend Builder, Startup CTO, or Privacy-First Builder. If uncertain, default to the Universal Objections section – those apply to everyone.
 2. Find their specific objection in the relevant persona section.
 3. Use the ACE response framework (Acknowledge, Counter, Evidence) to structure your response.
 4. Reference the evidence sources cited – do not improvise numbers or make unsourced claims. If the evidence does not exist for a specific claim, acknowledge the gap honestly.
 5. Consult the Severity Matrix (Section 7) to understand how frequently you will encounter each objection and how much it blocks adoption.
 6. Use the Quick Reference Card (Section 8) for at-a-glance one-line responses during live conversations.
-

2. Objection Response Framework: ACE

Every objection response in this playbook follows the **ACE** pattern. This ensures responses are empathetic, evidence-based, and verifiable – not dismissive or hand-wavy.

Acknowledge

Do not dismiss the concern. Validate that the developer’s hesitation is rational and based on real information. Dismissing objections (“That’s not a real concern”) destroys trust immediately. Instead, show that you understand why they hold this view and that reasonable people would reach the same conclusion given the information available.

Example: “That is a fair concern. Decentralized networks have historically struggled with reliability – Akash saw provider counts drop significantly.”

Counter

Provide a specific, measurable counterpoint. This must be a number, benchmark, architectural fact, or concrete comparison – not a vague reassurance. The counter should directly address the objection, not redirect to a different topic.

Example: “Gonka’s Sprint Consensus dedicates 98% of GPU compute to inference. Health-checked routing directs traffic only to nodes with proven sub-200ms latency. The architecture provides N-node redundancy where N is the number of active GPU hosts.”

Evidence

Point the developer to a source where they can verify the claim independently. Developers do not trust marketing claims – they trust things they can check themselves. If the evidence does not exist yet (e.g., no published uptime data), say so honestly.

Example: “The infrastructure code is open source at github.com/mitgor/gonka-ai-infrastructure. Sprint Consensus architecture is documented in the whitepaper, Section 3.”

3. Universal Objections (All Personas)

These objections are raised by every developer regardless of their persona, workload tier, or technical background. They represent the universal barriers to Gonka adoption and must be addressed before any persona-specific conversation can begin.

3.1 “I have never heard of Gonka.”

Frequency: Very High – nearly every first interaction **Severity:** High – blocks all subsequent evaluation

Acknowledge: Fair – Gonka is new to the OpenClaw ecosystem. If you have been using OpenRouter or Together AI, you would have no reason to encounter Gonka because it is not a built-in OpenClaw provider and has no presence in the standard provider comparison discussions. Your unfamiliarity is a function of our distribution, not our quality.

Counter: K2.5 scores 76.8% on SWE-Bench Verified – competitive with frontier models for agent workloads – at \$0.60/M input tokens (Gonka Scenario B pricing). You can try it with your existing OpenClaw configuration: three fields in `openclaw.json`, no new SDK, no wallet, no tokens. The switching cost is 90 seconds of editing a JSON file (competitive feature matrix: Section 3, K2.5 capabilities; message house: VP3, one URL change).

Evidence: - K2.5 benchmark results: SWE-Bench Verified 76.8%, stable across 200-300 sequential tool calls (competitive feature matrix: Section 3) - OpenClaw integration: 3 fields in `openclaw.json` – `baseUrl`, `apiKey`, `api: "openai-completions"` (message house: VP3, OpenClaw Configuration Pattern) - Integration test results: verified with OpenClaw, CrewAI, and LangGraph test suites (ARCHITECTURE.md: Component 6)

3.2 “Is this a crypto thing?”

Frequency: High – raised by 60-70% of Web2 developers at first contact **Severity:** High – immediate tab-close if the answer feels like “yes”

Acknowledge: Gonka’s infrastructure uses a token-incentivized GPU network, so the association is understandable. Many developers have had negative experiences with crypto-adjacent projects – hype without substance, wallet requirements, volatile token prices. Your skepticism is warranted based on how most crypto-infrastructure projects have presented themselves.

Counter: As a developer, you never interact with tokens, wallets, or blockchain. The developer experience is identical to OpenRouter or Together AI: API key authentication, USD pricing, standard OpenAI-compatible endpoints. You sign up with GitHub OAuth, get an API key, paste it into your config, and make inference calls. The infrastructure happens to be decentralized, which is why it is structurally cheaper (GPU hosts compete for your requests), but you never see or touch that layer (message house: vocabulary guidelines, Section 6; PITFALLS.md: Pitfall 2).

Evidence: - API docs showing standard auth flow: API key in header, no wallet connection, no on-chain transactions - Pricing in USD per million tokens, not in GNK or any other token denomination - Vocabulary guidelines: “wallet” never appears in developer-facing content; “API key” is the only auth term used (message house: Section 6.1, Never-Say List)

3.3 “K2.5 is not Claude or GPT.”

Frequency: High – most developers have muscle memory with Claude or GPT-4o **Severity:** Medium – addressable with benchmark data and practical demonstration

Acknowledge: K2.5 is newer and less known than GPT-4o or Claude. You have worked with those models, you know their strengths and quirks, and switching to an unfamiliar model feels risky – especially for production workloads. That hesitation is reasonable.

Counter: For agent workloads specifically – tool calling, multi-step reasoning, code generation – K2.5 is competitive at 5-10x lower cost. SWE-Bench Verified: 76.8%. Sequential tool calls: stable across 200-300 calls in testing. Context window: 131K tokens. K2.5 also has native Agent Swarm support for multi-agent workflows, which neither GPT-4o nor Claude offers natively. The question is not “is K2.5 as good as Claude on everything?” (it is not) – it is “is K2.5 good

enough for your agent’s specific tasks at 5-10x lower cost?” For most agent workloads, the answer is yes (competitive feature matrix: Section 3, Tool Calling; developer personas: model quality evidence).

Evidence: - K2.5 benchmark table: SWE-Bench 76.8%, 131K context, native tool calling (competitive feature matrix: Section 3) - Kimi K2.5 technical paper and public benchmarks - Integration test results showing K2.5 stability across 200-300 sequential tool calls (v1.2 test suite) - Cost comparison: K2.5 at \$0.50-0.60/M input vs GPT-4o at \$2.50/M input vs Claude Opus 4 at \$15/M input (pricing analysis: Section 4)

4. Weekend Builder Objections

The Weekend Builder is a solo developer running a personal AI agent as a side project on a \$20-50/month budget. They currently use OpenRouter because it is the default OpenClaw provider (zero configuration). Their primary decision driver is cost per task. They have never changed their inference provider and see no reason to unless the cost savings are dramatic and the switch is trivially easy (developer personas: Weekend Builder profile).

4.1 “OpenRouter already works, why switch?”

Frequency: High – the default objection from every Weekend Builder **Severity:** Medium – addressable with cost comparison if savings are dramatic enough

Acknowledge: OpenRouter’s zero-config setup is genuinely convenient. It is built into OpenClaw, requires no JSON editing, and provides access to 500+ models. If it works and the cost is tolerable, switching feels like unnecessary effort. You are right that “works” is a high bar to clear.

Counter: OpenRouter adds a 5.5% credit markup on every token. More importantly, OpenRouter is stateless – your agent resends its full context (9,600 tokens) on every heartbeat, 48 times per day. That is 460,800 tokens per day generating zero user-facing value. Gonka’s session persistence maintains your agent’s context server-side, reducing heartbeat token consumption by approximately 80%. At the Casual tier, that drops monthly costs from approximately \$52 (OpenRouter with markup) to approximately \$13 (Gonka Scenario B with sessions). One config change, 90 seconds of effort, \$39/month saved (pricing analysis: Sections 3 and 5; competitive feature matrix: Agent Sessions – OpenRouter LOSE; message house: Section 7.1, Weekend Builder competitive differentiation).

Evidence: - Price comparison: OpenRouter Casual tier ~\$52/month (including 5.5% markup) vs Gonka Scenario B ~\$13/month (pricing analysis: Section 5) - Heartbeat overhead: 44% of Casual tier tokens are heartbeats (pricing analysis: Section 3) - Session persistence: Gonka WIN, OpenRouter LOSE (competitive feature matrix: Section 1)

4.2 “But Gonka only has one model.”

Frequency: Medium – raised by developers who use OpenRouter’s model diversity **Severity:** Medium – valid concern but mitigatable with dual-provider approach

Acknowledge: Valid concern. OpenRouter offers 500+ models, and that breadth is genuinely useful. If K2.5 is unsuitable for a specific task – say, a particular type of translation or a niche domain reasoning task – there is no fallback model on Gonka. The competitive feature matrix scores Gonka as LOSE on model breadth, and that is an honest assessment (competitive feature matrix: Section 8).

Counter: For agent workloads, most developers use 1-2 models as their primary workhorse. K2.5 covers coding, reasoning, tool calling, and multi-step planning – the tasks that consume 80%+ of agent inference. If you need GPT-4o for structured outputs or Claude for long-context reasoning on specific tasks, keep OpenRouter configured as a secondary provider. OpenClaw supports multiple providers natively – use Gonka for the 80% of tasks where K2.5 handles it (and sessions save you money), and OpenRouter for the 20% where you need a different model. That is a net improvement over 100% on OpenRouter (developer personas: Weekend Builder, Objection #2; LangChain survey: 76% of teams use multiple models).

Evidence: - LangChain State of Agent Engineering: 76% of teams use multiple models, but a primary model handles the majority of calls - K2.5 capabilities: 76.8% SWE-Bench, 200-300 sequential tool calls, 131K context, Agent Swarm (competitive feature matrix: Section 3) - OpenClaw supports multiple provider configurations in the same `openclaw.json` (ARCHITECTURE.md: OpenClaw Configuration Pattern)

4.3 “What if K2.5 goes down?”

Frequency: Medium – raised by developers who have experienced OpenRouter free tier degradation **Severity:** Medium – addressable with dual-provider configuration

Acknowledge: Single-model risk is real. If K2.5 has a degradation or the Gonka network experiences an outage, your agent stops working with no fallback within the same provider. You have probably experienced OpenRouter’s free tier going unreliable during peak hours – you know how frustrating provider downtime is.

Counter: Keep your OpenRouter config as a fallback. OpenClaw supports multiple providers – use Gonka as primary (where session persistence saves you money) and OpenRouter as fallback (where model breadth gives you resilience). This is actually a net improvement over single-provider dependency on OpenRouter alone. If Gonka is down, your agent falls back to OpenRouter and continues working. If OpenRouter’s free tier degrades (which it does during peak hours), your agent is already on Gonka and unaffected. Dual-provider is strictly better than single-provider for resilience (developer personas: Weekend Builder, Objection #3).

Evidence: - OpenRouter free tier reliability issues: “models appear, disappear, hit throttles, degrade under peak load” (developer personas: Weekend Builder, Pain Points) - OpenClaw provider fallback support: multiple providers can be configured with fallback chains - Gonka uptime status: not yet published – this is an honest gap (competitive feature matrix: Section 7)

5. Startup CTO Objections

The Startup CTO is a technical co-founder running 3 agents across 6 channels on a \$200-500/month inference budget. Their primary decision driver is reliability and uptime – if the support agent goes down during a customer demo, they lose deals. They are cost-conscious but will not sacrifice reliability for savings. They are the hardest persona to convert because they require evidence, not promises (developer personas: Startup CTO profile).

5.1 “Decentralized means unreliable.”

Frequency: Medium – raised by CTOs who follow infrastructure trends **Severity:** Very High – primary blocker for production adoption

Acknowledge: Decentralized networks have historically struggled with consistency, and your concern is grounded in real precedent. Akash Network saw active providers drop below 100. Render Network’s daily active users declined below 100. Both suffered partly from reliability perception. If you have been following the decentralized compute space, you have seen more failures than successes (PITFALLS.md: Pitfall 3; competitive feature matrix: Section 7 – Gonka LOSE on uptime).

Counter: Gonka’s architecture is fundamentally different from raw GPU marketplaces. Sprint Consensus dedicates 98% of GPU compute to inference (versus 0% for traditional proof-of-work chains). Node health checking routes traffic only to nodes with proven sub-200ms latency – underperforming nodes are excluded from the routing pool automatically. The decentralized architecture provides N-node redundancy where N is the number of active GPU hosts, compared to M-datacenter redundancy where M is typically 2-4 for centralized providers. That said, this is architectural reasoning, not operational proof. We recommend a structured evaluation: deploy one agent on staging, measure p95 latency and uptime for one week, and compare to your current provider. Make the decision based on data, not architecture diagrams (message house: Section 7.2, Startup CTO competitive differentiation; ARCHITECTURE.md: Sprint Consensus architecture).

Evidence: - Sprint Consensus: 98% productive compute (ARCHITECTURE.md: USP #3) - Competitive feature matrix: Uptime / Reliability – Gonka LOSE (honest gap acknowledged) - Recommended evaluation path: staging deployment with measured metrics (developer personas: Startup CTO, Activation stage) - Infrastructure code: open source at github.com/mitgor/gonka-ai-infrastructure – audit the health checking and routing logic yourself

5.2 “No SLA, no deal.”

Frequency: Medium – raised by CTOs with production workloads and investor oversight **Severity:** Very High – hard blocker for production deployment

Acknowledge: Production workloads need guarantees, and your customers have SLAs with you. You need to pass reliability guarantees upstream to your infrastructure providers. A provider without an SLA creates an unquantifiable risk in your stack – and that is unacceptable when investors and customers are depending on uptime. This is not an unreasonable objection; it is a standard infrastructure procurement requirement.

Counter: This is an honest gap today. Gonka does not yet have a published SLA. We will not promise reliability we have not yet proven. For production use, we recommend a practical adoption path: start with non-critical workloads (dev/staging environments, cost-optimization secondary provider, internal tools) while we build the track record to back an SLA. Many infrastructure providers – including early-stage Vercel, Supabase, and Cloudflare Workers – launched without formal SLAs and built them as operational data accumulated. The question is whether the 73% cost savings on staging and internal workloads justify a limited initial deployment, not whether you should move all production traffic today (competitive feature matrix: Section 7; developer personas: Startup CTO, Objection #2; PITFALLS.md: Pitfall 3, recovery strategy).

Evidence: - Competitive feature matrix: Uptime / Reliability – Gonka LOSE, acknowledged - Recommended adoption path: staging first, then non-critical production, then gradual migration (developer personas: Startup CTO, AAARRRP journey, Activation to Retention) - Cost savings at Active tier: \$218/month (DeepInfra) vs \$59/month (Gonka Scenario B) – 73% reduction (pricing analysis: Section 6) - Transparent: acknowledge the gap, offer a practical path, do not overclaim

5.3 “We need compliance and data residency.”

Frequency: Low – raised by CTOs in regulated industries or serving enterprise customers **Severity:** High – can be a hard blocker for specific verticals (healthcare, finance, government)

Acknowledge: Decentralized inference raises legitimate data handling questions. If you are processing personally identifiable information, health data, or financial records, you need to know exactly where that data goes, who can access it, and under what jurisdictions it is stored. A decentralized network where prompts route to GPU hosts in unknown locations creates compliance uncertainty that no amount of cost savings can justify.

Counter: Gonka’s API gateway handles all request/response routing. Node operators receive encrypted payloads, not raw plaintext prompts. The gateway is the single point where routing decisions are made, which means geographic routing for data residency requirements is architecturally feasible (it is a gateway configuration, not a network redesign). Geographic routing is on the roadmap for data residency requirements. However, this is roadmap, not shipping – if you need data residency guarantees today, Gonka cannot provide them (message house: Section 4.2, Startup CTO positioning).

Evidence: - Gateway architecture: centralized routing with encrypted payloads to nodes (ARCHITECTURE.md: architecture-to-message mapping, API gateway) - Current limitation: geographic routing is roadmap, not implemented - Honest assessment: if compliance is a hard requirement today, Gonka is not ready. Use a provider with published data residency guarantees (OpenAI, Anthropic) for compliant workloads

6. Privacy-First Builder Objections

The Privacy-First Builder is a senior developer or security engineer building agents that process sensitive data (legal, medical, financial, politically sensitive content). They currently self-host vLLM or use Ollama for sensitive workloads. Their primary decision driver is data privacy and censorship resistance – they need a provider that does not log prompts, does not filter outputs, and does not have a content policy team reviewing conversations. They are technically sophisticated and will verify every privacy claim against the source code (developer personas: Privacy-First Builder profile).

6.1 “How decentralized is it really?”

Frequency: High – the first question from every privacy-motivated developer **Severity:** High – determines whether they proceed with evaluation

Acknowledge: Many projects claim decentralization but have central points of control. You are right to be skeptical. “Decentralized” has been used as a marketing term by projects that are architecturally centralized – a load balancer in front of two data centers is not decentralization. Your question is about the actual architecture, not the marketing claims.

Counter: Gonka’s Sprint Consensus runs on independently operated GPU nodes. Node operators run their own hardware, join the network voluntarily, and are incentivized by host earnings (compute rewards) for serving inference. The compute layer is genuinely distributed across independent operators. The API gateway is currently centralized – this is an honest architectural reality. The gateway handles authentication, routing, and encryption, which means it is a central point of control today. The roadmap includes gateway decentralization, but that is future work. Current state: distributed compute layer, centralized gateway. That is more decentralized than OpenAI or Together AI (fully centralized) but less decentralized than a fully peer-to-peer network (message house: Section 4.3, Privacy-First Builder; developer personas: Privacy-First Builder, Objection #1).

Evidence: - Infrastructure code: open source at github.com/mitgor/gonka-ai-infrastructure – audit the gateway, node communication, and routing logic yourself - Network architecture: independently operated GPU nodes with centralized API gateway - Honest limitation: gateway is centralized today; gateway decentralization is roadmap - Comparison: more decentralized than OpenAI/Together AI (centralized), less than fully peer-to-peer networks

6.2 “Can node operators see my prompts?”

Frequency: High – critical question for any privacy-sensitive workload **Severity:** Very High – determines whether sensitive workloads can use Gonka

Acknowledge: This is the critical question for privacy-sensitive workloads, and you should not trust any provider that dismisses it. If node operators can see plaintext prompts, the entire privacy value proposition collapses. You are right to demand a precise architectural answer, not a vague reassurance.

Counter: The API gateway encrypts payloads between the gateway and the inference node. Node operators serve inference but do not have access to plaintext prompts in the standard communication flow. However – and this is the honest part – the encryption is transport-level (TLS between gateway and node), not computation-level. The node’s GPU processes plaintext prompts during inference execution. A malicious node operator with low-level access to GPU memory could theoretically intercept prompt data during processing. TEE (Trusted Execution Environment) support is on the roadmap to provide computation-level isolation, but it is not built today. Current state: no central log aggregation (architectural privacy), not end-to-end encrypted inference (cryptographic privacy) (developer personas: Privacy-First Builder, Objection #1; PITFALLS.md: Pitfall 1; message house: VP4, honesty note).

Evidence: - Gateway encryption: TLS between gateway and inference nodes - Current limitation: TEE not yet implemented; privacy is architectural, not cryptographic - The message house explicitly states: “Current privacy guarantee is architectural (no central log aggregation), not cryptographic (TEE-based encrypted inference is not yet built). This distinction must be stated clearly in all privacy-related messaging.” (message house: VP4, honesty note) - Infrastructure code: audit the gateway encryption at github.com/mitgor/gonka-ai-infrastructure

6.3 “Why not just run my own vLLM?”

Frequency: Medium – raised by developers who already self-host or are considering it **Severity:** Medium – addressable with total cost of ownership comparison

Acknowledge: Self-hosting gives you maximum control. You own the hardware, you control the network, you see every log file, and no third party ever touches your prompts. If maximum privacy is your non-negotiable requirement and budget is secondary, self-hosting is the most conservative choice. There is no provider – including Gonka – that matches the privacy guarantees of hardware you physically control.

Counter: Self-hosting K2.5 (a 1T parameter MoE model) requires significant GPU resources – at minimum 4x H100 GPUs (\$120K+ hardware purchase or \$15K+/month cloud rental). Beyond the GPU cost: electricity (\$500-1,500/month for 4x H100s at residential rates), cooling infrastructure, hardware maintenance, OS and driver updates, vLLM version

management, and the engineering time to handle all of it. Gonka gives you the same model with similar privacy guarantees (no content filtering, no central logging) at per-token pricing. The break-even point is approximately \$15K/month in inference spend – if you spend less than that, Gonka’s per-token pricing is cheaper than the total cost of ownership of self-hosting. Additionally, self-hosted vLLM does not include session persistence, automatic model tiering, memory API, or webhook notifications – features that Gonka provides as part of the managed service (developer personas: Privacy-First Builder, Gonka Value Proposition; message house: Section 4.3, Privacy-First positioning).

Evidence: - GPU cost for K2.5 self-hosting: 4x H100 at \$30K each = \$120K+ hardware, or \$15K+/month cloud GPU rental (ARCHITECTURE.md: self-hosting cost analysis) - Gonka pricing: Scenario B at \$0.35/\$1.75 per 1M tokens (pricing analysis: Section 4 – note: Gonka pricing is TBD, this is scenario-based) - Feature gap in self-hosting: no sessions, no tiering, no memory, no webhooks (competitive feature matrix: feature comparison; message house: Privacy-First Builder positioning) - Break-even analysis: ~\$15K/month inference spend is the crossover point where self-hosting becomes cheaper than per-token pricing

7. Objection Severity Matrix

All objections ranked by frequency (how often you will hear it) and severity (how much it blocks adoption). Use this matrix to prioritize which objections to address in marketing materials, landing pages, and community interactions.

Objection	Frequency	Severity	Persona(s)	Response Priority	Resolution Strategy
“I have never heard of Gonka.”	Very High	High	All	P0	Solve with ecosystem presence: Reddit, Hacker News, OpenClaw Discord, ClawHub
“Is this a crypto thing?”	High	High	All (especially Weekend Builder, Startup CTO)	P0	Solve with messaging: landing page that looks like Vercel, no crypto jargon
“OpenRouter already works, why switch?”	High	Medium	Weekend Builder	P1	Solve with cost comparison: heartbeat savings calculator, \$52 vs \$13
“K2.5 is not Claude or GPT.”	High	Medium	Weekend Builder, Startup CTO	P1	Solve with benchmarks: SWE-Bench 76.8%, tool calling stability, cost comparison
“No SLA, no deal.”	Medium	Very High	Startup CTO	P0	Honest gap – mitigate with staged adoption path (staging first, then gradual)
“Decentralized means unreliable.”	Medium	Very High	Startup CTO	P0	Mitigate with staging test recommendation, publish uptime data when available
“How decentralized is it really?”	High	High	Privacy-First Builder	P1	Solve with open-source code audit, honest architecture description
“Can node operators see my prompts?”	High	Very High	Privacy-First Builder	P0	Honest answer: architectural privacy today, TEE on roadmap
“Gonka only has one model.”	Medium	Medium	Weekend Builder, Privacy-First Builder	P1	Mitigate with dual-provider approach (Gonka primary + OpenRouter fallback)
“What if K2.5 goes down?”	Medium	Medium	Weekend Builder	P2	Mitigate with dual-provider fallback configuration
“We need compliance / data residency.”	Low	High	Startup CTO	P2	Honest gap – geographic routing on roadmap, not available today
“Why not just run my own vLLM?”	Medium	Medium	Privacy-First Builder	P1	Solve with TCO comparison: \$15K/month break-even + missing agent features

Priority Legend

- **P0:** Must be addressed in all first-touch materials (landing page, quickstart, first community interaction). These objections block the funnel at the widest point.
- **P1:** Address in secondary materials (comparison blog posts, documentation, FAQ pages). These objections slow evaluation but do not prevent initial engagement.
- **P2:** Address in deep-dive content (technical architecture docs, detailed FAQ, one-on-one conversations). These objections affect specific scenarios and can be handled contextually.

Key Insight from Severity Matrix

The two highest-severity objections – “No SLA” and “Can node operators see my prompts?” – both require honest acknowledgment of current gaps, not clever counterarguments. For the Startup CTO, the honest answer is “not yet, here is our path to an SLA.” For the Privacy-First Builder, the honest answer is “architectural privacy today, cryptographic privacy is on the roadmap.” Attempting to spin these gaps into strengths will be detected and will destroy credibility with technically sophisticated audiences (PITFALLS.md: Pitfall 1; message house: VP4, honesty note).

8. Quick Reference Card

One page. Pin it. Use it in live conversations, Discord replies, and community interactions.

Weekend Builder – Top 3 Objections

Objection	One-Line Response
“OpenRouter already works.”	“It works, but 44% of your bill is heartbeats. Gonka sessions cut that to ~9% – \$52/month drops to ~\$13.”
“Only one model.”	“K2.5 handles 80% of agent tasks. Keep OpenRouter as fallback for the other 20% – dual-provider is better than single.”
“What if it goes down?”	“Keep OpenRouter as fallback. Dual-provider is strictly more resilient than single-provider dependency.”

Startup CTO – Top 3 Objections

Objection	One-Line Response
“Decentralized = unreliable.”	“Test on staging for a week. Measure p95 latency and uptime. Decide based on your data, not our claims.”
“No SLA.”	“Honest gap today. Start with staging and internal tools – the 73% cost savings justify limited deployment while we build the track record.”
“Need multiple models.”	“Use Gonka for K2.5 workloads (sessions save 73%). Keep your current provider for tasks needing other models. Net savings, not a replacement.”

Privacy-First Builder – Top 3 Objections

Objection	One-Line Response
“How decentralized really?”	“Compute: genuinely distributed. Gateway: centralized today, decentralization on roadmap. Code is open source – audit it.”
“Can operators see prompts?”	“Transport encryption: yes. Compute isolation (TEE): not yet. Current privacy is architectural, not cryptographic. We state this honestly.”
“Why not self-host?”	“Self-hosting K2.5 costs \$15K+/month for GPUs alone, with no sessions, tiering, or webhooks. Gonka: same model, similar privacy, agent features included.”

Universal – Top 3 Objections

Objection	One-Line Response
“Never heard of Gonka.”	“K2.5 at 76.8% SWE-Bench, \$0.60/M tokens, session persistence that cuts heartbeat costs 80%. Try it in 90 seconds – 3 fields in openclaw.json.”
“Is this crypto?”	“No. API key, USD pricing, OpenAI-compatible endpoint. You never touch tokens, wallets, or blockchain.”

Objection	One-Line Response
"K2.5 is not Claude/GPT."	"For agent workloads: 76.8% SWE-Bench, 200-300 tool calls stable, 5-10x cheaper. Not better at everything – better value for agents."

9. Response Anti-Patterns

What NOT to do when handling objections. These anti-patterns destroy credibility and should be actively avoided by anyone representing Gonka.

Anti-Pattern 1: Dismissing the objection

Bad: "That is not really a concern. Decentralized networks are the future." **Why bad:** Tells the developer their rational concern is invalid. They stop listening immediately. **Instead:** Acknowledge first, then counter with evidence.

Anti-Pattern 2: Overclaiming on privacy

Bad: "Your prompts are completely private on Gonka. No one can see them." **Why bad:** TEE is not built. Node operators process plaintext prompts on their GPUs. This claim is technically false today, and security-minded developers will discover this and never trust you again (PITFALLS.md: Pitfall 1). **Instead:** "Current privacy is architectural (no central log aggregation), not cryptographic (TEE is on the roadmap). We state this distinction clearly."

Anti-Pattern 3: Using crypto jargon in responses

Bad: "Gonka's tokenomics ensure deflationary pressure on inference costs as the network's staking requirements increase validator participation." **Why bad:** Every word after "Gonka's" is on the never-say list. The developer hears "crypto scam" and leaves (message house: Section 6.1, Never-Say List). **Instead:** "Inference gets cheaper as more GPU providers join the network and compete for your requests."

Anti-Pattern 4: Comparing to all competitors simultaneously

Bad: "Gonka is better than OpenRouter, Together AI, OpenAI, and Anthropic because..." **Why bad:** Shotgun comparisons are not credible. No provider is better than all competitors on all dimensions. **Instead:** Compare to the single competitor the developer is most likely using: Weekend Builder vs OpenRouter, Startup CTO vs Together AI, Privacy-First vs Akash (message house: Section 7, per-persona competitor focus).

Anti-Pattern 5: Making promises about unbuilt features

Bad: "We will have an SLA by Q3." **Why bad:** Roadmap commitments to individual developers create expectations that, if unmet, destroy trust permanently. **Instead:** "An SLA framework is on our roadmap. We will publish it when we have the operational data to back it. Until then, we recommend staging deployment."

Document: gonka_objection_playbook.md / Version 1.0 / 2026-04-01 Sources: gonka_message_house.md, gonka_developer_persona.md, gonka_competitive_feature_matrix.md, gonka_agent_pricing_analysis.md, ARCHITECTURE.md, PITFALLS.md Feeds into: Phase 18 (Channel Strategy), Phase 19 (Partnership & Ecosystem)